

Pontifícia
Universidade
Católica do
Rio de Janeiro

Relatório - Trabalho 1

INF1316 - Sistemas Operacionais

Simulando o escalonamento preemptivo com chamadas de sistema

Judy Faria - 2312668

Luís Fernando Bessa Seibel
Departamento de Informática

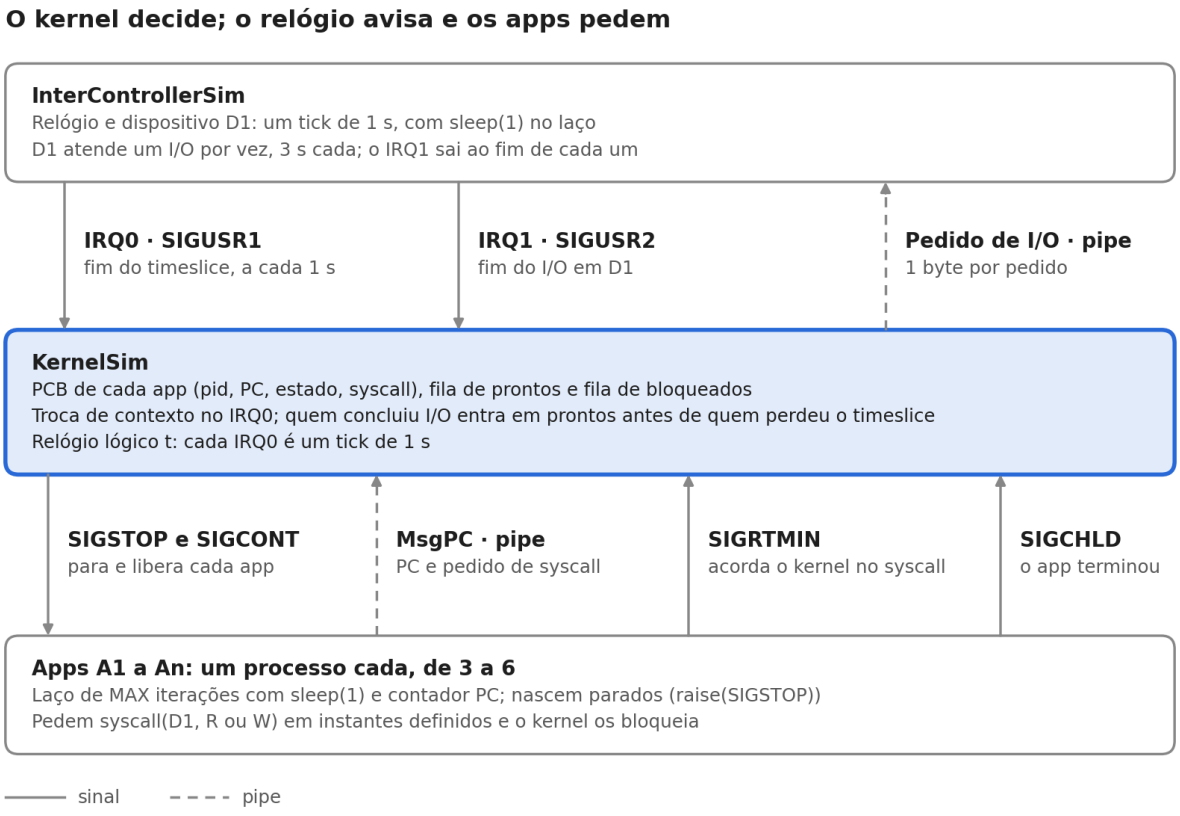
Rio de Janeiro, 17 de Outubro de 2026

Pontifícia
Universidade
Católica do
Rio de Janeiro

Visão geral

O KernelSim escalona de 3 a 6 aplicações a cada 1 s, bloqueia as que pedem I/O em D1 e as libera quando o InterControllerSim avisa o fim do I/O.

Cada elemento da Figura 1 do enunciado é um processo Unix: o kernel cria os apps e o InterController com `fork` e `exec`, e todos conversam só por sinais e pipes.



Leitura do diagrama: o InterController manda o IRQ0 a cada tick e o IRQ1 ao fim de cada I/O. O kernel manda `SIGSTOP` e `SIGCONT` aos apps, e os apps respondem por pipe e por sinal. Só o kernel escolhe quem roda.

Pontifícia
Universidade
Católica do
Rio de Janeiro

Arquivos entregues e como compilar e executar

t1.c → KernelSim: escalonamento, PCBs, filas de prontos e de bloqueados, tratamento de IRQ0, IRQ1, syscall e término.

intercontroller.c → InterControllerSim: gera o IRQ0 a cada 1s e o IRQ1 ao fim de cada I/O em D1 (3s por pedido).

app.c → Aplicação Ax: laço de `MAX` iterações com `sleep(1)`, contador `PC` e `syscall(D1, R/W)`.

`common.h` → `MAX e a estrutura MsgPC trocada pelo pipe.`

`teste_fila.c` → `Teste isolado da fila de prontos (Etapa 4).`

`saidas/` → `Saídas completa de cada cenário`

Compilação e execução, na mesma pasta:

```
gcc -o app app.c
gcc -o interController intercontroller.c
gcc -DN=3 -o t1 t1.c        # N = número de apps, de 3 a 6
./t1                    # tabela de syscalls padrão
./t1 2R 2W 9R               # um token por app: <PC><R|W>; "-" = sem syscall
```

O kernel é infinito: depois que todos os apps terminam, avisa uma vez e fica ocioso até o Ctrl+C, que encerra todos os processos.



Pontifícia
Universidade
Católica do
Rio de Janeiro

Resultado final

Com 3 e com 6 apps, todos os processos terminam com `PC final = 9`, ou seja, executam exatamente `MAX` = 10 iterações.

A saída completa é longa e fica em `saidas/` (um arquivo por cenário): `teste1_normal.txt` para 3 apps e `teste4_seis_apps.txt` para 6 apps. Cada linha do kernel tem a forma `[t=10s] IRQ0 | A6 rodando | Prontos: A2 A3 A4 A1 A5 | Bloqueados: | PCs: A1=2 A2=1 ...`, ou seja, o instante, o evento, quem roda, as duas filas e o PC de cada processo.

O que a execução com 6 apps mostra:

- **Round Robin de 1 s:** o processo em execução gira A1, A2, A3, ... um por tick, e a fila de prontos acompanha.
- **Syscall bloqueia:** ao pedir I/O, o app é parado pelo kernel (`SIGSTOP`), vai para `Bloqueados` e o próximo roda na mesma hora.
- **Prioridade do I/O:** em t=10, o A1 acabou de concluir o I/O e entra em `Prontos` na frente do A5, que perdeu o timeslice no mesmo tick.
- **Contexto preservado:** o A1 bloqueou com `PC`=2 e, ao voltar, imprimiu `PC`=3; o PC nunca volta a 0.
- **Todos terminam:** cada app termina com `PC final = 9`; o kernel avisa uma vez e fica ocioso até o Ctrl+C.

O D1 atende um I/O por vez, 3 s cada. O A3 e o A4 pediram I/O no mesmo instante (t=16), e foram atendidos em t=20 e t=23:

App	**Pediu I/O em t**	**Concluiu em t**
A1	6	10
A2	11	15
A3	16	20
A4	16	23
A5	22	26
A6	23	29

Quando o pedido seguinte chega depois de o D1 ficar livre, o D1 espera ocioso (A1 para A2 e A2 para A3: 5 s entre as conclusões). Nunca há menos de 3 s entre duas conclusões.

Com 3 apps, a saída mostra também o caso em que todos estão bloqueados: aparece `Nenhum processo rodando` e o primeiro IRQ1 despacha o A1 na mesma hora, sem esperar o próximo IRQ0.



Pontifícia
Universidade
Católica do
Rio de Janeiro

Como foi desenvolvido

Uso de inteligência artificial

Usei o assistente de IA Claude, da Anthropic, como tutor durante o desenvolvimento deste trabalho. Eu já dominava os conceitos de escalonamento das listas, mas não sabia como traduzi-los em código C com `fork`, sinais e pipes. Por isso pedi que a IA me guiasse em etapas, explicando cada conceito, cada trecho de código, cada teste e o porquê, em vez de entregar o trabalho pronto.

O processo seguiu um ciclo: a IA explicava o conceito e propunha um passo pequeno; eu escrevia o código, compilava, rodava e mostrava o resultado; ela apontava erros e explicava a causa, e eu corrigia. Ao fim de cada etapa fazíamos uma recapitulação, que usei como base para a aba correspondente deste relatório. Algumas decisões de projeto foram minhas, com a IA apresentando as alternativas, como o D1 serial, o relógio lógico em ticks e o kernel infinito.

Em partes em que travei, pedi o código e a IA o forneceu com explicação, como a troca de contexto, a comunicação por pipe, o término dos processos e o `sigsuspend`. A IA também montou o script de testes automatizados e o diagrama de arquitetura, e organizou a aba principal deste relatório a partir das minhas abas e das saídas das execuções. Executei e testei tudo na minha máquina.

O trabalho foi construído em sete etapas, cada uma testada antes da seguinte; cada aba traz objetivo, código, resultado e conclusão.

Etapa 1 → `fork` e `exec`: três apps como processos filhos, cada um sabendo seu número por `argv; Os três rodam em paralelo e a ordem é do Linux: é isso que o kernel deve substituir.` (https://github.com/JudyFaria/SO/commit/f9683031a9e0e791129004fcb8909a5bb7c74e67)

Pontifícia
Universidade
Católica do
Rio de Janeiro

Etapa 2 → SIGSTOP e SIGCONT pelo pai; o app nasce parado (raise); espera com WUNTRACED; O PC sobrevive à pausa; o sleep mede relógio, não CPU.
(https://github.com/JudyFaria/SO/commit/d747be7ec0cafdc0c1858e293bdced6e1a5ec564)

Etapa 3 → InterController como processo à parte (IRQ0 = SIGUSR1); handler, flag e pause() no kernel; O kernel só reage; o timeslice passou para o sleep do InterController.
(https://github.com/JudyFaria/SO/commit/44f250985bd012e9e5ea0d2a082bd5a9d65d2725)

Etapa 4 → PCB, fila de prontos, troca de contexto no IRQ0; PC informado por pipe; O PCB guarda um retrato do momento da parada.
(https://github.com/JudyFaria/SO/commit/1b76eb7fb19faed07466927a12f11ffcb178d29c)

Etapa 5 → Syscall, fila de bloqueados, IRQ1, prioridade do I/O; D1 serial; I/O de 3 s por pedido, em ordem de chegada, na frente do timeslice.
(https://github.com/JudyFaria/SO/commit/0a4c0aa0df1af84b6c2a9ee5d172fbf5ea43c6dd)

Etapa 6 → Término dos apps (SIGCHLD e WNOHANG); sigprocmask e sigsuspend; Exatamente MAX iterações por app e nenhuma corrida no pause().
(https://github.com/JudyFaria/SO/commit/96f5f520349ccc22065f51ee2adf51123bfc315e)



Pontifícia
Universidade
Católica do
Rio de Janeiro

Etapa 7 → Tempo decorrido em cada linha, N de 3 a 6, tabela de syscalls por argv, kernel infinito com Ctrl+C limpo; A linha do tempo mostra as filas e o PC; os cenários rodam automaticamente.

Testes → Sete cenários automáticos, rodados por roda_testes.sh, conferem os requisitos do enunciado sem depender de ler a saída à mão.

(https://github.com/JudyFaria/SO/commit/8cc1dce89d0a44e6b95434c473a49728f2a7a17a)

Etapa 1

Objetivo: criar 3 processos filhos que executam o programa `app`, cada um sabendo quem é (A1, A2, A3). Ainda sem controle de nenhum tipo.

Feito:

- **main.c** (futuro KernelSim): um *for* que dá *fork( )* 3 vezes e guarda os PIDs no array *pids[ ].* Cada filho vira o *app* via execl. No final, o pai mata e recolhe cada filho com *kill(SIGKILL)* + *waitpid.*

- **app.c** (futuro Ax): lê o próprio número em *argv[ ]* e roda um loop infinito imprimindo o número e o PC.

Conceitos:
fork( ) : Cria uma cópia do processo. Retorna **0 no filho** e o **PID do filho no pai**.
exec( ) : Troca o programa do processo, mas o PID continua o mesmo. Só retorna se **falhar**.
argv[ ] : É assim que o pai passa informação ao filho no exec. Só aceita texto, daí o *snprintf*.
kill( ) : Envia qualquer sinal, não só mata.
waitpid( ) : O pai recolhe o filho, evitando processos zumbis.

Erros que apareceram:

1. O *waitpid* travaria para sempre num app com laço infinito, e foi o que aconteceu. Então tive que fazer o pai enviar SIGKILL antes para cada filho.

Resultado:

```
@JudyFaria
A1 PC=0
A2 PC=0
A3 PC=0
A1 PC=1
A3 PC=1
A2 PC=1
A1 PC=2
A3 PC=2
A2 PC=2
```

→ ordem variou, mostrando o escalonador Linux trabalhando.

Conclusão:
É exatamente no ponto mostrado no Resultado que o trabalho entra, tendo como objetivo substituir essa decisão tomada pelo escalonador do Linux pelo KernelSim que será criado, controlando quem roda e quando, via SIGSTOP/SIGCONT.

Etapa 2

Objetivo: tirar do Linux a decisão de quem roda e entregá-la ao pai, que faz o papel de kernel. Só um app roda por vez, usando SIGSTOP/SIGCONT.

Feito:

- **app.c :** Para o app nascer congelado e começar somente após receber um SIGCONT, ele recebeu um *raise(SIGSTOP)* antes do loop.

- **main.c** : Agora com 4 blocos:
  1. Laço de criação dos forks filhos (etapa 1);
  2. Laço com *waitpid(pids[i], NULL, WUNTRACED)*, para esperar os três pararem;
  3. Laço do escalonador: *SIGCONT, sleep(1), SIGSTOP e atual = (atual + 1)%N*, por 9 turnos;
  4. Laço final com SIGKILL e waitpid.

Conceitos:

- raise(SIGSTOP) : O processo envia um sinal para si mesmo. Faz o app nascer parado, sem depender da velocidade do pai.

- Waitpid com WUNTRACED : Retorna quando o filho para, não só quando termina. É a confirmação de que ele realmente parou.

Resultado:

(a)

```
@JudyFaria
A1 PC=0
A2 PC=0
A3 PC=0
A1 PC=1
A1 PC=2
A2 PC=1
A3 PC=1
A1 PC=3
A2 PC=2
A3 PC=2
@JudyFaria
```

→ linha extra

(b)

```
@JudyFaria
A1 PC=0
A2 PC=0
A3 PC=0
A1 PC=1
A2 PC=1
A3 PC=1
A1 PC=2
A2 PC=2
A3 PC=2
```

Conclusão:

1. O PC sobrevive à pausa. O SIGSTOP congela o processo inteiro, memória incluída, vemos isso no fato de que o A1 contou 0 → 1 → 2 → 3 sem zerar.

2. O *sleep* mede tempo de relógio, não de CPU (sleep do app.c). O cronômetro continua correndo com o app parado. Quando o SIGCONT chega, o sleep já expirou e retorna na hora, então o app imprime logo no começo da fatia.

3. Corrida no fim da fatia. O *sleep(1)* do pai e o do app expiram quase no mesmo instante. Se o pai ganha, sai uma linha por turno (imagem (b) do Resultado). Se o app ganha, sai uma extra (imagem (a) do Resultado).

Etapa 3

Objetivo: separar os papéis que o pai acumulava na Etapa 2. O InterControllerSim vira um processo à parte que gera IRQ0 a cada 1s, e o KernelSim deixa de contar tempo e passa a reagir ao sinal. Ainda sem apps, apenas os dois processos.

Feito:

- **intercontroller.c** : um laço infinito com *sleep(1)* e *kill(pid_kernel, SIGUSR1)*. O Pid do kernel vem de *getppid( )*.

- **main.c** (kernel):
    1. Registra o handler de SIGUSR1 com *sigaction;*
    2. Cria o InterController com um único *fork* e *exec;*
    3. Entra num laço com *pause( )* que ao acordar, testa a flag, zera e imprime uma mensagem.
    4. No fim, *kill e waitpid* no InterController.

Conceitos:

- O **`SIGUSR1`** e o **`SIGUSR2`** são os sinais de usuário: o sistema não lhes dá significado nenhum, e o programador decide o que cada um quer dizer. Por isso servem para emular interrupções:
    - SIGUSR1 vai ser o seu IRQ0 (fim do timeslice)
    - SIGUSR2 vai ser o seu IRQ1 (fim do I/O em D1)

  Sem o handler registrado, a ação padrão encerra o processo.

- **sigaction**: registra qual função o sistema chama quando o sinal chega. O handler precisa estar registrado antes de o InterController começar a enviar .

- **Handler**: Interrompe o programa em qualquer ponto, então quase nada é segur nele.

- Flag **volatile sig_atomic_t**: o handler só levanta um aviso, e o trabalho real é feito no laço principal. O volatile impede o compilador de guardar a flag num registrador e nunca enxerga a mudança. O tipo, `sig_atomic_t` garante que ler ou escrever a flag é uma operação indivisível, sem risco de o handler interromper a escrita pela metade.

- **pause( )**: Suspende o processo até chegar qualquer sinal que tenha handler.

- **getppid( )**: Pid do pai. Funciona porque o kernel é o pai direto do InterController, e o *exec* preserva o pai.

Resultado:



Conclusão:
O teste simples não prova que zerar a flag é necessário. Como o único sinal que chega é o SIGUSR1, a versão sem zeramento também funciona. O bug só apareceria quando outros sinais acordassem o *pause()*.
Agora o timeslice está no *sleep(1)* do InterController, agora o kernel só espera o IRQ0.

Etapa 4

Objetivo: o kernel passa a decidir quem roda com base no IRQ0, mantendo o contexto (PC) de cada processo. Feito em quatro sub-etapas: 4a - PCB e fila de prontos; 4b - criar os apps e preencher os PCBs; 4c - troca de contexto no IRQ0 e 4d - o app informa o PC por pipe.

Feito:

- **PCB** (pid, num_app (id), pc) e o array *pcbs[N]*, uma ficha por app.

- **Fila de prontos** circular, de índices, com *enfileira_prontos, desenfileira_prontos* (devolve -1 se vazia) e *imprime_estado.*

- **Inicialização:** um pipe único antes dos forks, criação dos apps, PCBs preenchidos pelo pai, espera com *WUNTRACED*, todos na fila, o primeiro recebe *SIGCONT*, e só então nasce o InterController.

- **Laço Principal:** *pause( )*, e a cada IRQ0 a troca de contexto, com o PC atualizado a partir de pipe.

- *Common.h* com MsgPC, e o *app* escrevendo *{id, pc}* no pipe a cada iteração.

Conceitos:

PCB : A ficha do processo no kernel, onde o contexto é guardado.

Troca de Contexto : 1 - *SIGSTOP* no atual; 2 - recolocá-lo no fim de prontos; 3 - tirar o próximo da fila; 4 - *SIGCONT* nele.

PC do app : A variável do app só muda quando ele executa, e o *SIGSTOP* a preserva. O PC do kernel é um espelho.

Pipe : O descritor é herdado no fork e sobrevive ao exec. É passado ao app pelo *argv*, como o número do app; Um canal só serve os três apps, e escritas pequenas são atômicas, então não se misturam.

O_NONBLOCK : O *read* devolve -1 com o pipe vazio, em vez de travar o kernel.

Sinais assíncronos : waitpid(WUNTRACED) depois do SIGSTOP; o *kill* só envia. Esperar a parada garante que o app não escreve mais nada antes de o kernel ler.

Erros que apareceram:

1. Fila sem checar vazia: O *tamanho* ficou negativo e a fila continuou “funcionando” com dados velhos do array.

Resultado:

```
@JudyFaria →/workspaces/SO/Trabalho_G1 (main) $ ./t1
A1 rodando | Prontos: A2 A3  | PCs: A1 = 0 A2 = 0 A3 = 0
A1 PC=0
A1 PC=1
A2 rodando | Prontos: A3 A1  | PCs: A1 = 1 A2 = 0 A3 = 0
A2 PC=0
A3 rodando | Prontos: A1 A2  | PCs: A1 = 1 A2 = 0 A3 = 0
A3 PC=0
A3 PC=1
A1 rodando | Prontos: A2 A3  | PCs: A1 = 1 A2 = 0 A3 = 1
A1 PC=2
A2 rodando | Prontos: A3 A1  | PCs: A1 = 2 A2 = 0 A3 = 1
A2 PC=1
A3 rodando | Prontos: A1 A2  | PCs: A1 = 2 A2 = 1 A3 = 1
A3 PC=2
A1 rodando | Prontos: A2 A3  | PCs: A1 = 2 A2 = 1 A3 = 2
A1 PC=3
A2 rodando | Prontos: A3 A1  | PCs: A1 = 3 A2 = 1 A3 = 2
A2 PC=2
A3 rodando | Prontos: A1 A2  | PCs: A1 = 3 A2 = 2 A3 = 2
A3 PC=3
A1 rodando | Prontos: A2 A3  | PCs: A1 = 3 A2 = 2 A3 = 3
A1 PC=4
A2 rodando | Prontos: A3 A1  | PCs: A1 = 4 A2 = 2 A3 = 3
A2 PC=3
A3 rodando | Prontos: A1 A2  | PCs: A1 = 4 A2 = 3 A3 = 3
A3 PC=4
A1 rodando | Prontos: A2 A3  | PCs: A1 = 4 A2 = 3 A3 = 4
A1 PC=5
```

Conclusão:

1. O kernel controla: O “rodando” gira A1, A2, A3, e a fila acompanha.
2. O PC sobrevive à pausa, e o PC do kernel passou a refletir o do app.
3. O PCB guarda um **retrato do momento da parada**, e não o valor ao vivo. Que é o “salvar o contexto”.

Etapa 5

Objetivo: fazer os apps pedirem I/O (`syscall(D1, R/W)`), com o kernel bloqueando-os numa fila própria e o IRQ1 liberando-os, na ordem do enunciado. Feito em 5 sub-etapas.

5a: decisões de projeto

- Como o app avisa o kernel: Pipe para os dados (app, dispositivo, R/W) mais o sinal **`SIGRTMIN`** só para acordar o *`pause()`*. O pipe sozinho não acorda o kernel.
- Quem para o app no syscall: O **kernel**, com `SIGSTOP`, como diz o enunciado. A auto-parada (`raise`) foi considerada e descartada.
- Formato da mensagem: `MsgPC` ganhou o campo `tipo` (`MSG_PC` ou `MSG_SYSCALL`), `dispositivo` e `operacao`.
- Instantes dos syscalls: A1 em PC=2 (R), A2 em PC=2 (W), A3 em PC=5 (R). Foram escolhidos para a fila de bloqueados chegar a ter mais de um processo.
- IRQ1: Começou **periódico** (a cada 3s) e foi revisado no fim da etapa (veja a revisão).

5b: o IRQ1 isolado

Implementado: handler de `SIGUSR2` com a flag `irq1_pendente`, e o kernel imprimindo `IRQ1 recebido`. No InterController, um contador de **ticks** de 1s, com o IRQ1 nos múltiplos de 3.

Conceito: tick é uma batida do relógio, e uma fonte de tempo só decide tudo. Os eventos mais lentos saem do contador com `%`.

Resultado:

```
A1 rodando | Prontos: A2 A3  | PCs: A1 = 1 A2 = 0 A3 = 0

IRQ1 recebido

A1 PC=2
A2 rodando | Prontos: A3 A1  | PCs: A1 = 2 A2 = 0 A3 = 0
A2 PC=1
A3 rodando | Prontos: A1 A2  | PCs: A1 = 2 A2 = 1 A3 = 0
A3 PC=1
A1 rodando | Prontos: A2 A3  | PCs: A1 = 2 A2 = 1 A3 = 1

IRQ1 recebido

A1 PC=3
A2 rodando | Prontos: A3 A1  | PCs: A1 = 3 A2 = 1 A3 = 1
A2 PC=2
A3 rodando | Prontos: A1 A2  | PCs: A1 = 3 A2 = 2 A3 = 1
A3 PC=2
A1 rodando | Prontos: A2 A3  | PCs: A1 = 3 A2 = 2 A3 = 2

IRQ1 recebido
```

→ Uma linha `IRQ1 recebido` a cada 3 trocas, com o rodízio intacto.

Decisão de ordem: o bloco do **IRQ1 vem antes do IRQ0** no laço, e o InterController envia o SIGUSR2 antes do SIGUSR1. Por conta da prioridade de quem terminou I/O sobre quem perdeu o timeslice descrito no enunciado, com isso ele entrar na fila **antes** do preemptado. A ordem de envio é uma **mitigação**, não uma garantia.

5c: duas filas com o mesmo código

Implementado: *struct* `Fila { itens[N], inicio, tamanho }`, e as funções recebendo ponteiro (*`enfileira(Fila *f, ...)`*, *`desenfilera(Fila *f)`*). `prontos` e `bloqueados` são duas instâncias. O PCB ganhou o `estado` (`PRONTO`, `RODANDO`, `BLOQUEADO`). O *`imprime_estado`* mostra as duas filas.

Decisão: o tamanho `N` basta para as duas, porque cada processo está em **um lugar só** (rodando, prontos ou bloqueados). Esse invariante foi usado como critério de correção em toda a etapa.

Resultado: comportamento igual ao anterior, com *`Bloqueados`* **:** sempre vazio.

```
@JudyFaria →/workspaces/SO/Trabalho_G1 (main) $ ./t1
A1 rodando | Prontos: A2 A3  | Bloqueados:  | PCs: A1=0 A2=0 A3=0
A1 PC=0
A1 PC=1
A2 rodando | Prontos: A3 A1  | Bloqueados:  | PCs: A1=1 A2=0 A3=0
A2 PC=0
A3 rodando | Prontos: A1 A2  | Bloqueados:  | PCs: A1=1 A2=0 A3=0
A3 PC=0

IRQ1 recebido

A1 rodando | Prontos: A2 A3  | Bloqueados:  | PCs: A1=1 A2=0 A3=0
A1 PC=2
A2 rodando | Prontos: A3 A1  | Bloqueados:  | PCs: A1=2 A2=0 A3=0
A2 PC=1
A3 rodando | Prontos: A1 A2  | Bloqueados:  | PCs: A1=2 A2=1 A3=0
A3 PC=1
A3 PC=2

IRQ1 recebido

A1 rodando | Prontos: A2 A3  | Bloqueados:  | PCs: A1=2 A2=1 A3=2
A1 PC=3
A2 rodando | Prontos: A3 A1  | Bloqueados:  | PCs: A1=3 A2=1 A3=2
A2 PC=2
A3 rodando | Prontos: A1 A2  | Bloqueados:  | PCs: A1=3 A2=2 A3=2
A3 PC=3
```

5d: o syscall

Implementado:

- *common.h* com `TipoMsg` e a `MsgPC` nova.
- PCB com `dispositivo`, `operacao` e `pediu_syscall`.
- Terceiro handler (`SIGRTMIN`) com a flag `syscall_pendente`.
- `atualiza_pcs` distingue os dois tipos de mensagem: PC atualiza o PCB, syscall guarda os parâmetros e marca `pediu_syscall`.
- App: escreve a mensagem de syscall **antes** de enviar o sinal, para o pedido já estar no pipe quando o kernel acordar.

Decisão central: quem decide a fila é a **mensagem**, e o sinal só acorda o kernel. O `SIGRTMIN` não diz quem pediu. Se o IRQ0 acordasse o kernel antes, ele leria a mensagem de syscall dentro do *`atualiza_pcs`* e a marca *`pediu_syscall`* evitaria que o processo fosse para **prontos**. Por isso o bloco do IRQ0 também escolhe a fila pela marca.

Outras decisões: quando um processo bloqueia, o próximo é despachado **na hora**, sem esperar o IRQ0. Efeito colateral: o novo processo recebe uma fatia mais curta, porque o relógio não reinicia.

Resultado:

```
A3 rodando | Prontos: A2  | Bloqueados: A1  | PCs: A1=2 A2=1 A3=1
A3 PC=2
A2 rodando | Prontos: A3  | Bloqueados: A1  | PCs: A1=2 A2=1 A3=2
A2 PC=2
A3 rodando | Prontos:  | Bloqueados: A1 A2  | PCs: A1=2 A2=2 A3=2
A3 PC=3
A3 PC=4
A3 rodando | Prontos:  | Bloqueados: A1 A2  | PCs: A1=2 A2=2 A3=3

IRQ1 recebido

A3 rodando | Prontos:  | Bloqueados: A1 A2  | PCs: A1=2 A2=2 A3=4
A3 PC=5
Nenhum processo rodando | Prontos:  | Bloqueados: A1 A2 A3  | PCs: A1=2 A2=2 A3=5
Nenhum processo rodando | Prontos:  | Bloqueados: A1 A2 A3  | PCs: A1=2 A2=2 A3=5
Nenhum processo rodando | Prontos:  | Bloqueados: A1 A2 A3  | PCs: A1=2 A2=2 A3=5

IRQ1 recebido

Nenhum processo rodando | Prontos:  | Bloqueados: A1 A2 A3  | PCs: A1=2 A2=2 A3=5
```

Os três bloqueiam em PC=2, PC=2 e PC=5, o PC fica salvo (nunca volta a 0) e aparece o caso **“nenhum processo rodando”**.

5e: o IRQ1 desbloqueia

Implementado: no bloco do IRQ1, *`liberado = desenfilera(&bloqueados)`*; se *`>= 0`*, o processo vai para **prontos**, sem SIGCONT. Se *`rodando < 0`*, despacha o próximo de prontos na hora (*`SIGCONT, estado = RODANDO, imprime_estado`*).

Resultado: prioridade do I/O:

```
A2 PC=1
A3 rodando | Prontos: A2  | Bloqueados: A1  | PCs: A1=2 A2=1 A3=0
A3 PC=1
A2 rodando | Prontos: A3  | Bloqueados: A1  | PCs: A1=2 A2=1 A3=1
A2 PC=2
A3 rodando | Prontos:  | Bloqueados: A1 A2  | PCs: A1=2 A2=2 A3=1
A3 PC=2
A1 rodando | Prontos: A3  | Bloqueados: A2  | PCs: A1=2 A2=2 A3=2
A1 PC=3
A3 rodando | Prontos: A1  | Bloqueados: A2  | PCs: A1=3 A2=2 A3=2
A3 PC=3
A1 rodando | Prontos: A3  | Bloqueados: A2  | PCs: A1=3 A2=2 A3=3
A1 PC=4
A3 rodando | Prontos: A2 A1  | Bloqueados:  | PCs: A1=4 A2=2 A3=3
A3 PC=4
A2 rodando | Prontos: A1 A3  | Bloqueados:  | PCs: A1=4 A2=2 A3=4
A2 PC=3
A1 rodando | Prontos: A3 A2  | Bloqueados:  | PCs: A1=4 A2=3 A3=4
A1 PC=5
A3 rodando | Prontos: A2 A1  | Bloqueados:  | PCs: A1=5 A2=3 A3=4
A3 PC=5
```

← O A2, recém-liberado, está **na frente** do A1, que acabara de perder o timeslice.

Revisão: o IRQ1 passou de periódico para serial

O problema: o teste com `NUM_IRQS = 20` mostrou que o tempo de I/O variava. O IRQ1 era um relógio fixo de 3s, sem relação com o pedido.

Processo	Bloqueou	Liberado	Espera
A1	tick 3	tick 6	3s
A2	tick 8	tick 9	1s
A3	tick 14	tick 15	1s

O enunciado diz que D1 precisa de **3 segundos por operação**, então dois dos três esperaram menos que isso.

A solução, implementada: D1 virou um **dispositivo serial**. Atende um pedido por vez, 3s cada, e o seguinte espera.

- **IC com estado:** *`pedidos`* (o em curso mais os que esperam) e *`restante`* (ticks que faltam). A cada tick: lê o pipe, desconta o I/O em curso e envia o IRQ1 quando *`restante`* chega a 0. Se D1 está livre e há pedido, carrega *`restante = 3`*. Só depois envia o IRQ0.

- **Pipe de I/O:** o kernel escreve **1 byte por pedido**. Sinais comuns se fundem e perdem pedidos próximos. Com `O_NONBLOCK` marcado antes do fork do InterController, o relógio dele não trava esperando pedidos.

- **Função *`bloqueia`*:** marca `BLOQUEADO`, enfileira e avisa o IC. Estava duplicada no syscall e no IRQ0, e agora é um lugar só. As filas do kernel e a contagem do IC não divergem.

- **Pontas dos pipes fechadas** por quem não as usa.

Resultado:

```
A3 PC=1
A2 rodando | Prontos: A3  | Bloqueados: A1  | PCs: A1=2 A2=1 A3=1
A2 PC=2
A3 rodando | Prontos:  | Bloqueados: A1 A2  | PCs: A1=2 A2=2 A3=1
A3 PC=2
A3 rodando | Prontos:  | Bloqueados: A1 A2  | PCs: A1=2 A2=2 A3=2   ←
A3 PC=3
Nenhum processo rodando | Prontos:  | Bloqueados: A1 A2 A3  | PCs: A1=2 A2=2 A3=3
A1 rodando | Prontos:  | Bloqueados: A2 A3  | PCs: A1=2 A2=2 A3=3
A1 PC=3
A1 rodando | Prontos:  | Bloqueados: A2 A3  | PCs: A1=3 A2=2 A3=3
A1 rodando | Prontos:  | Bloqueados: A2 A3  | PCs: A1=3 A2=2 A3=3
A1 PC=4
A1 rodando | Prontos:  | Bloqueados: A2 A3  | PCs: A1=4 A2=2 A3=3   ←
A1 PC=5
A2 rodando | Prontos: A1  | Bloqueados: A3  | PCs: A1=5 A2=2 A3=3
A2 PC=3
A1 PC=6
A1 rodando | Prontos: A2  | Bloqueados: A3  | PCs: A1=5 A2=3 A3=3
A2 rodando | Prontos: A1  | Bloqueados: A3  | PCs: A1=6 A2=3 A3=3
A2 PC=4
A1 rodando | Prontos: A3 A2  | Bloqueados:  | PCs: A1=6 A2=4 A3=3
A1 PC=7
A3 rodando | Prontos: A2 A1  | Bloqueados:  | PCs: A1=7 A2=4 A3=3
```

Os IRQ1 saíram espaçados de **exatamente 3 ticks** (7, 10, 13), em FIFO, e o primeiro IRQ1 despachou o A1 na hora, com todos bloqueados.

Processo	**Bloqueou**	**Liberado**	**Observação**
A1	após o tick 3	tick 7	o IC o descobre no tick 4 e conta 3
A2	após o tick 5	tick 10	D1 ocupado com o A1 até o tick 7
A3	após o tick 6	tick 13	D1 ocupado até o tick 10

→ **Conceitos da etapa:**

- **Syscall:** O pedido do app ao kernel (aqui simulado). Não tem relação com o `syscall()` do Linux.
- **IRQ1:** O **fim** do I/O, avisado pelo dispositivo (o IC). Syscall é o pedido e IRQ1 é o término.
- **Bloqueio:** I/O é lento, então o processo sai da CPU e da fila de prontos, e outro roda.
- **Sinal acorda, pipe informa:** O pipe carrega dados e o sinal acorda o *`pause()`*. Os dois se complementam.
- **Sinais comuns se fundem:** Dois pedidos próximos virariam um aviso. Por isso o pedido de I/O ao InterController vai por pipe, byte a byte.
- **Marca no PCB (`pediu_syscall`):** A decisão da fila vem da mensagem, e não de qual sinal chegou primeiro.
- **Ordem dos blocos:** IRQ1, syscall, IRQ0: o I/O tem prioridade, e o syscall precisa ser visto antes de o IRQ0 preemptar.
- **FIFO nas duas filas:** A ordem de saída do IC coincide com a de chegada ao pipe e com a fila *`bloqueados`*, por construção.

Limitações:

- **O *pause()* tem uma corrida:** um sinal que chegue entre o teste da flag e a chamada do *pause()* só é visto no próximo acordar.

- **Granularidade de 1s:** o InterController descobre o pedido no tick seguinte, então o I/O dura de 3 a 4s para o processo.

- **Fatia quase zero após o despacho pelo IRQ1:** se o IRQ0 do mesmo acordar do *pause()* estiver pendente, o processo recém-despachado é preemptado quase na hora.

- **O app imprime antes da linha do kernel** às vezes. A linha do kernel é a decisão, e a do app é a evidência.

Etapa 6

Objetivo: fechar o ciclo de vida dos processos (o app termina e o kernel percebe e reage) e eliminar a corrida do *pause()*. Teve duas sub-etapas, 6a - término) e 6b - *sigprocmask* + *sigsuspend*, mais o refactor *despacha()*.

6a: término de processos

Implementado:

- `MAX = 10` em `common.h`, e o laço do app limitado a `MAX` iterações.
- Estado `TERMINADO` no `enum`, com `(fim)` no `imprime_estado`.
- `SIGCHLD` com handler que liga *filho_terminou*, registrado com `SA_NOCLDSTOP`.
- *verifica_terminados()*: *waitpid(..., WNOHANG)* em cada PCB ainda vivo.
- *finaliza(id)*: lê o último PC do pipe, marca `TERMINADO`, zera *pediu_syscall*, conta em *terminados* e libera *rodando*. É o único lugar que trata o término.
- *para_e_salva(id)*: `SIGSTOP`, `waitpid(WUNTRACED)` e `WIFSTOPPED(st)`. Devolve 1 se parou e 0 se o processo já tinha terminado (e então chama *finaliza()*).
- *despacha()*: tira o próximo de prontos, marca `RODANDO` e manda `SIGCONT`. Substituiu quatro cópias do mesmo trecho.
- `rodando` e `terminados` viraram globais, porque as funções auxiliares os alteram.

Decisões:

- **Como o kernel descobre o término:** `SIGCHLD` mais `waitpid(WNOHANG)`: o sinal diz que algum filho mudou, e o `waitpid` por PCB diz qual.
- **`SA_NOCLDSTOP`:** Sem ele o `SIGCHLD` chegaria também quando um filho para ou continua, e o kernel acordaria por motivo falso a cada troca.

- **Parou ou terminou:** O `SIGSTOP` pode chegar a um processo que já morreu. O `waitpid(WUNTRACED)` então devolve status de saída, e o `WIFSTOPPED` distingue.

- **Quem trata o término:** Uma função só (`finaliza`), chamada por quem o encontrar primeiro.

- **Quem pode terminar:** Só o processo **rodando**: um bloqueado está parado por `SIGSTOP`. Por isso `finaliza` só limpa o `rodando`, sem mexer nas filas.

- **Ordem dos blocos:** Término, IRQ1, syscall, IRQ0. O término vem primeiro para o IRQ0 nunca tentar parar um processo morto.

- **Fim do programa:** O enunciado diz que o kernel é infinito. O `break` em `terminados == N` é **só para teste**: sem ele, o kernel fica ocioso no `sigsuspend`.

6b: `sigprocmask` e `sigsuspend`

O problema: entre o teste da flag e o `pause()` existe uma janela. Um sinal que chega ali liga a flag, mas o kernel dorme sem vê-la.

Implementado:

- os quatro sinais (`SIGUSR1`, `SIGUSR2`, `SIGRTMIN`, `SIGCHLD`) ficam **bloqueados**, e o *`pause()`* virou *`sigsuspend(&mascara_vazia)`* no fim do laço.

Decisões:

- **O bloqueio vem depois de criar todos os filhos.** A máscara é herdada pelo *`fork`* e sobrevive ao *`exec`*. Bloquear antes faria os apps e o IC nascerem bloqueados.

- **O `sigsuspend` fica no fim do laço**, depois de testar todas as flags. Sinais que chegaram durante o tratamento ficaram pendentes e são entregues dentro dele.

- **O `break` vem antes do `sigsuspend`**.

→ **Conceitos da etapa:**

- **`Sigset_t:`** Só uma **lista** de sinais (um formulário). `sigemptyset` esvazia e `sigaddset` acrescenta. Sozinha não bloqueia nada.

- **Máscara do processo:** Estado que o **sistema** guarda por processo. O `sigprocmask` copia a sua lista para ele, e depois a variável já cumpriu o papel.

- **Sinal bloqueado:** Não chama o handler, mas **não se perde**: fica pendente.

- **`sigsuspend(&mascara_vazia):`** Guarda a máscara **do processo**, instala a vazia, dorme, e ao acordar restaura a original. A `mascara_vazia` é só a máscara temporária.

- **Atômico:** Indivisível: desbloquear e dormir são uma operação só. Um sinal que já estava pendente é entregue na hora, e o `sigsuspend` nem chega a dormir.

- **`WNOHANG:`** `waitpid` que não trava: devolve 0 se o filho ainda vive.

- **`WIFSTOPPED(st):`** Distingue “parou” de “terminou” no status do `waitpid`.

- **`MAX:`** O **trabalho total** de cada processo, em iterações (cada uma com `sleep(1)`): o equivalente ao tempo de CPU total. Não é o timeslice, e o kernel não corta o processo nele.

→ **Testes realizados (após a Etapa 6):**

- Mudando só a tabela *`pcs_syscall`* do `app.c`:

1) Caminho Normal:

```
int pcs_syscall[] = {2, 2, 3}; // instantes do pedido e A1, A2 e A3
```

```
...
A2 PC=9
A3 rodando | Prontos: A2  | Bloqueados:  | PCs: A1=9(fim) A2=9 A3=9
A3 terminou (PC final = 9)
A2 rodando | Prontos:  | Bloqueados:  | PCs: A1=9(fim) A2=9 A3=9(fim)
A2 terminou (PC final = 9)
Nenhum processo rodando | Prontos:  | Bloqueados:  | PCs: A1=9(fim) A2=9(fim) A3=9(fim)
Todos os processos terminaram.
```

→ resultado: Os três terminam com `PC final = 9`.

2)Syscall na última iteração:

```
int pcs_syscall[] = {2, 2, 9}; // instantes do pedido e A1, A2 e A3
```

```
...
A1 rodando | Prontos: A3 A2  | Bloqueados:  | PCs: A1=6 A2=5 A3=8
A1 PC=7
A3 rodando | Prontos: A2 A1  | Bloqueados:  | PCs: A1=7 A2=5 A3=8
A3 PC=9
A2 rodando | Prontos: A1  | Bloqueados: A3  | PCs: A1=7 A2=5 A3=9
A2 PC=6
A1 rodando | Prontos: A2  | Bloqueados: A3  | PCs: A1=7 A2=6 A3=9
A1 PC=8
A2 rodando | Prontos: A1  | Bloqueados: A3  | PCs: A1=8 A2=6 A3=9
A2 PC=7
A1 rodando | Prontos: A2  | Bloqueados: A3  | PCs: A1=8 A2=7 A3=9
A1 PC=9
A2 rodando | Prontos: A3 A1  | Bloqueados:  | PCs: A1=9 A2=7 A3=9
A2 PC=8
A3 rodando | Prontos: A1 A2  | Bloqueados:  | PCs: A1=9 A2=8 A3=9
A3 terminou (PC final = 9)
```

→ resultado: O A3 bloqueia em PC=9, volta e termina. O contexto (A3=9) sobrevive ao bloqueio.

3) Os três bloqueados no começo:

```
int pcs_syscall[] = {1, 1, 1}; // instantes do pedido e A1, A2 e A3
```

```
…
A3 PC=0
A2 rodando | Prontos: A3  | Bloqueados: A1  | PCs: A1=1 A2=0 A3=0
A2 PC=0
A3 rodando | Prontos: A2  | Bloqueados: A1  | PCs: A1=1 A2=0 A3=0
A3 PC=1
A2 rodando | Prontos:  | Bloqueados: A1 A3  | PCs: A1=1 A2=0 A3=1
A2 PC=1
Nenhum processo rodando | Prontos:  | Bloqueados: A1 A3 A2  | PCs: A1=1 A2=1 A3=1
A1 rodando | Prontos:  | Bloqueados: A3 A2  | PCs: A1=1 A2=1 A3=1
A1 rodando | Prontos:  | Bloqueados: A3 A2  | PCs: A1=1 A2=1 A3=1
A1 PC=2
A1 rodando | Prontos:  | Bloqueados: A3 A2  | PCs: A1=2 A2=1 A3=1
A1 PC=3
A1 rodando | Prontos:  | Bloqueados: A3 A2  | PCs: A1=3 A2=1 A3=1
A1 PC=4
…
```

→ resultado: FIFO, D1 serial, despacho imediato e prioridade do I/O.

Testes realizados

Sete cenários automáticos, rodados por `roda_testes.sh`, conferem os requisitos do enunciado sem depender de ler a saída à mão.

Cenário	Tabela de syscalls	Apps	O que exercita
`teste1_normal`	`2R 2W 3R`	3	Caminho normal
`teste2_syscall_ultima`	`2R 2W 9R`	3	Syscall na última iteração: o app bloqueia, volta e termina
`teste3_todos_bloqueados`	`1R 1W 1R`	3	Os três bloqueados no começo: CPU ociosa, despacho pelo IRQ1, D1 serial
`teste4_seis_apps`	`2R 2W 3R 3W 4R 4W`	6	Seis processos
`teste5_sem_syscall`	`- - -`	3	Round Robin puro, sem I/O
`teste6_seis_bloqueados`	`1R 1W 1R 1W 1R 1W`	6	Seis apps pedindo I/O juntos: fila de bloqueados quase cheia
`teste7_encerramento`	padrão	3	Kernel infinito: segue vivo e ocioso, e encerra tudo no Ctrl+C e no `SIGTERM`, até com apps parados

Para cada cenário, o script confere na saída:

- cada app executa exatamente `MAX` iterações;
- o aviso de fim aparece e o Ctrl+C encerra o kernel (cenários 1 a 6);
- dois fins de I/O ficam separados por no mínimo 3 ticks (D1 serial);
- o tempo `t` nunca anda para trás;
- em toda linha, rodando, prontos, bloqueados e terminados somam `N`: cada processo está em um lugar só;
- nenhuma linha do kernel aparece misturada com a de um app;
- nenhum processo sobra depois do teste.

→ **Uso, na pasta do trabalho:**

```
SRC=t1.c ./roda_testes.sh                          # todos os cenários
./roda_testes.sh teste3                            # só os que começam com "teste3"
./roda_testes.sh --confere saidas/teste4_seis_apps.txt 6   # só confere uma saída já gerada
```

A saída de cada cenário fica em `saidas/`, e é a evidência citada na seção Resultado final.